

CS 498: Machine Perception

HW0: Perspective, Homography, and Epipolar Geometry

Fall 2026

Course weight	10%
Work mode	Individual
Release	Friday, September 4, 2026
Due	Friday, September 18, 2026 at 11:59 p.m. America/Chicago

1 What you will build

Do you enjoy our class so far? Would you like more people across campus to know about it? In this homework, we investigate how to promote CS498: Machine Perception to a broader audience through *virtual advertising*.

Imagine that the CS498 logo has been painted directly onto the State Farm Center floor during a basketball broadcast. Your job is to synthesize that effect from a single photograph: the logo must have the correct metric size, follow the court perspective, and look attached to the floor rather than pasted onto the screen. You will then lift from the 2D court plane into 3D, recover a camera projection and pose from known landmarks, and place a Stanford bunny at center court. Finally, you will reuse the same homography code on a tennis broadcast and estimate epipolar geometry between two images.



(a) Basketball: metric planar insertion.



(b) Tennis bonus: logo composition with players.

Figure 1: Target visual results. Your required basketball output should match the floor perspective; the tennis bonus additionally preserves dynamic foreground occlusion.

The starter keeps the graded surface small. You edit five functions and one tennis metric-coordinate block. Each task file contains its own readable `main()` pipeline; low-level image loading, warping, projection, and plotting utilities are provided in `hw0_utils.py`.

Use the course computing environment, which provides Python 3.10 or newer, NumPy, OpenCV, ImageIO, and Matplotlib. You do not need to install packages or use a separate runner or test framework.

2 Run one file per task

Work in the `handout/` directory. Each task runs directly and writes its figures and metrics to a matching folder under `outputs/`:

```
python task1_homography.py
python task2_projection.py
python task3_epipolar.py
```

Edit only the clearly marked block between “Modify your code below” and “DO NOT MODIFY CODE OUTSIDE THE BLOCK” in each task file. Read that file’s `main()` from top to bottom to see where your functions are called. Each subtask prints and saves its own checkpoint, so implement one subtask, rerun the file, and inspect that checkpoint before continuing. Runnable placeholders keep the later checkpoints working while you proceed. Do not edit `hw0_utils.py`.

File	Student work
<code>task1_homography.py</code>	<code>estimate_homography</code> , <code>logo_to_image_homography</code> , <code>alpha_blend</code> , <code>tennis XY rows</code>
<code>task2_projection.py</code>	<code>estimate_projection_matrix</code>
<code>task3_epipolar.py</code>	<code>estimate_fundamental_matrix</code>

Coordinate conventions. Image points use $(u, v) = (\text{column}, \text{row})$ with an upper-left origin. Court coordinates are meters: court point 3 is the origin, the long edge is $+x$, the short edge is $+y$, and $+z$ points upward. Do not reorder supplied matches or landmarks.

3 Basketball scene and metric world frame

The image supplies four court corners (points 0–3) and four backboard corners (points 4–7). The top-down diagram below fixes the metric world frame before any matrix is estimated. The court is 28.65 m by 15.24 m; point 3 is $(0, 0, 0)$; and the backboard lies in the plane $x = 1.22$ m, is 1.83 m wide, and spans $z = 2.745$ m to $z = 3.965$ m.

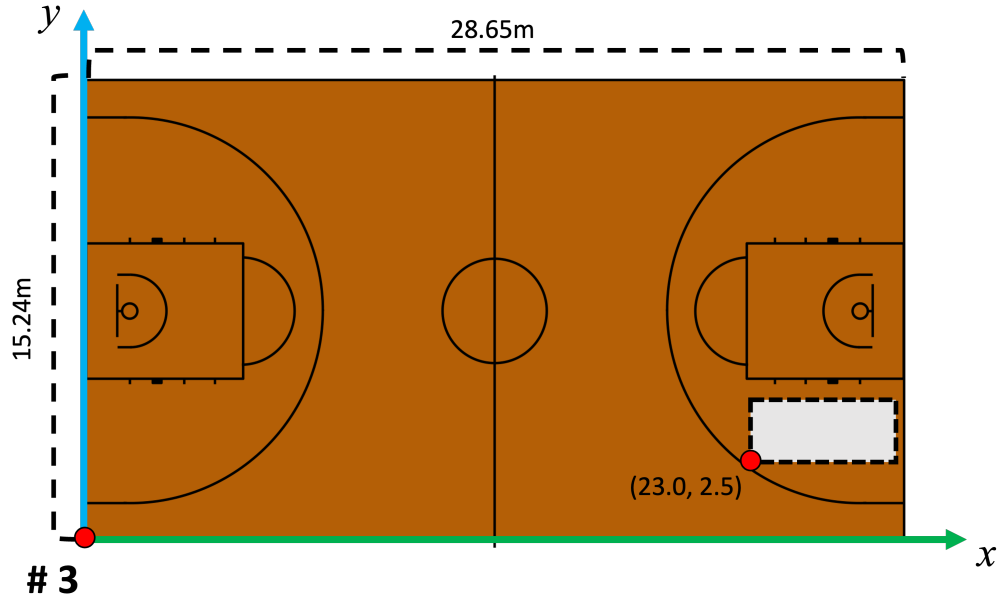


Figure 2: Top-down State Farm Center coordinate frame, landmark identities, and the required logo placement. Backboard point pairs overlap in this view because their x, y coordinates match and only their heights differ.

There are two related geometric estimates. The four $z = 0$ court corners determine the planar homography used for the logo. They do *not* determine a general camera matrix. Adding the four elevated backboard corners provides non-coplanar 3D–2D correspondences for the 3×4 projection matrix P . After estimating P , the provided `decompose_projection` helper writes

$$P \sim K[R \mid t], \quad C = -R^\top t,$$

where K is upper triangular, R maps world coordinates to camera coordinates, t is the world-to-camera translation, and C is the camera optical center in the metric court frame.

4 Task 1: Homography and 2D insertion [3.4 points]

1A. Estimate the homography [1.0 point]

Implement `estimate_homography` in `task1_homography.py`. Given $N \geq 4$ matching court points (x, y) and image pixels (u, v) , return a 3×3 matrix H for which

$$\lambda[u, v, 1]^\top = H[x, y, 1]^\top.$$

Normalize each point set by translating its centroid to the origin and scaling its mean distance to $\sqrt{2}$. Build the $2N \times 9$ DLT system, solve its right null space with SVD, denormalize, and choose a stable scale.

1B. Place and composite the basketball logo [0.9 point]

Implement `logo_to_image_homography`. Compose the transforms from logo pixels to a 6 m by 3 m court rectangle and then through H to the image. Account for the fact that logo pixel v points down while court y points up. The provided code performs the bilinear inverse warp.

Implement `alpha_blend`. For foreground RGB I_{logo} , background I_{court} , and logo alpha $\alpha \in [0, 1]$, compute

$$I_{\text{out}} = \alpha I_{\text{logo}} + (1 - \alpha) I_{\text{court}}.$$

Checkpoint 1A produces `task1a_basketball_homography.png`; Checkpoint 1B produces `task1b_basketball_insertion.png`. Check that the projected corners match and that the logo follows the court perspective without a rectangular screen-space border.

1C. Recover tennis court coordinates and reuse the code [0.9 point]

This part asks you to recover the metric XY coordinates corresponding to eight provided image points. The starter already supplies `TENNIS_UV`. In `task1_homography.py`, fill only the eight rows of `TENNIS_XY` in the labeled A–H order shown below. For each row:

1. read the metric (x, y) location from the regulation-court diagram;
2. verify that the letter names the same visible court intersection in frame 0 of `data/tennis/match005.mp4`; and
3. record only that metric (x, y) value in the matching row of `TENNIS_XY`.

Do not change the provided UV values or the row order. Then run the same three functions used for basketball. Do not write a second homography estimator or a second image-compositing implementation.

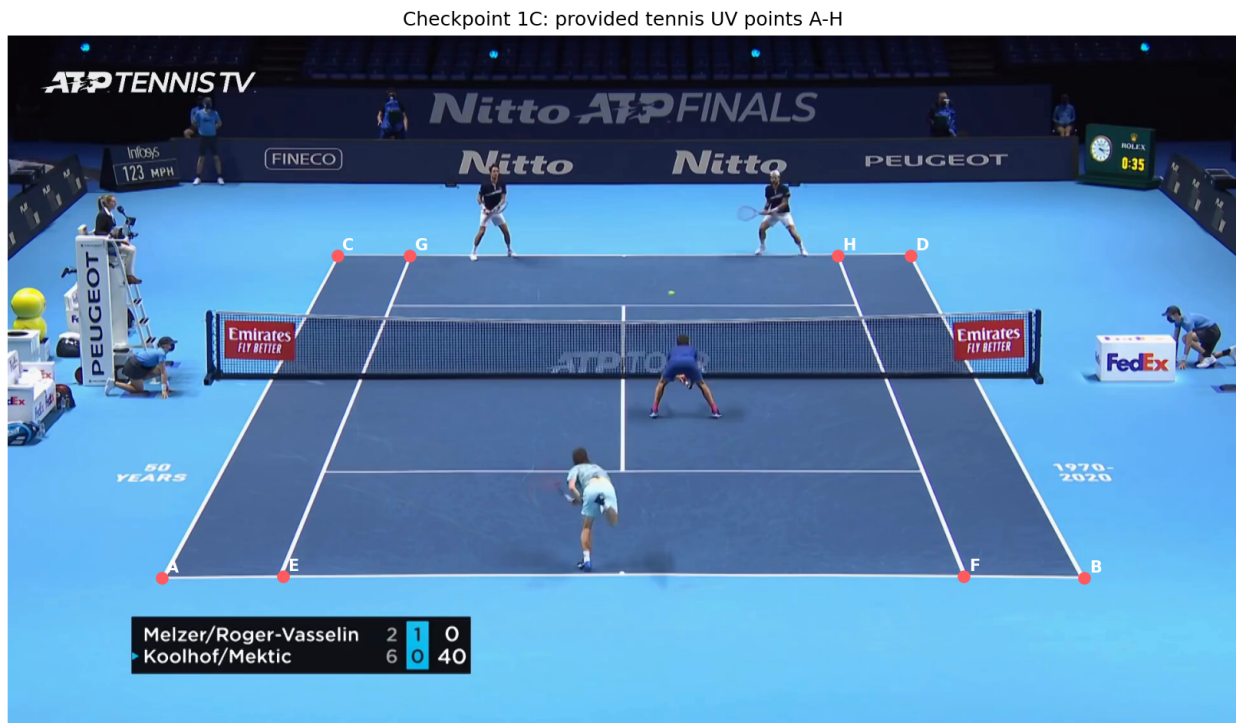


Figure 3: The eight tennis intersections A–H. Their image UV values are provided; you enter only their metric XY values in `task1_homography.py`.

Use the true regulation dimensions shown below: court length 23.77 m, doubles width 10.97 m, and singles width 8.23 m. For our course world frame, the origin is the center of the near baseline, $+x$ runs across the court, and $+y$ points toward the far baseline. The virtual logo occupies $[-3, 3] \times [0.5, 3.5]$ meters.

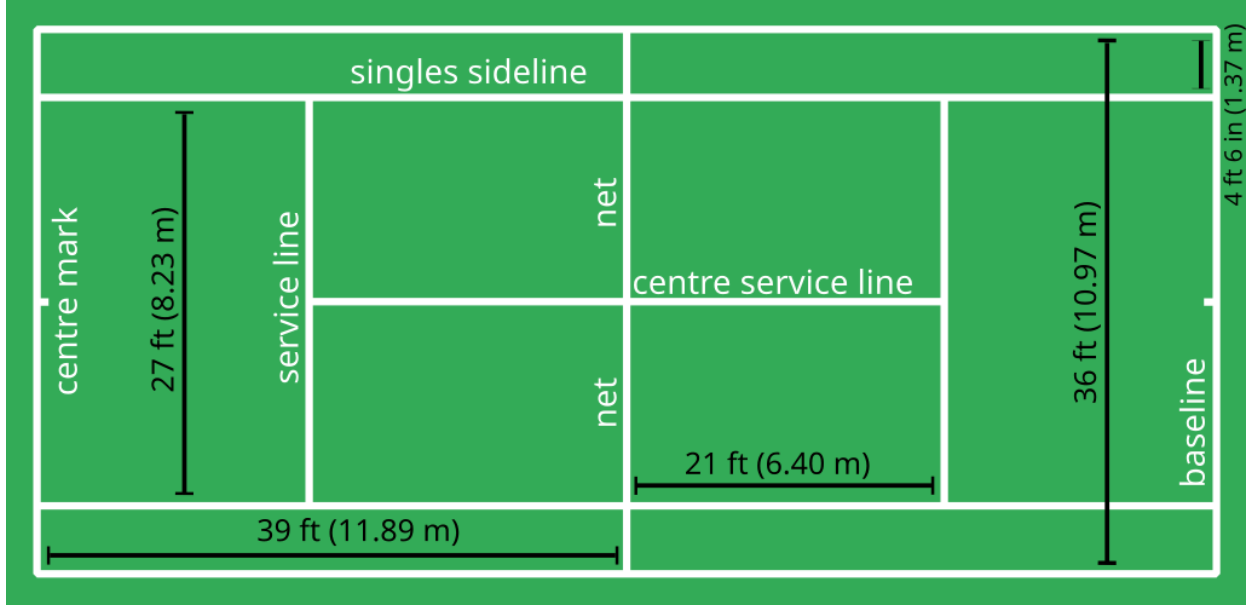


Figure 4: Regulation tennis-court dimensions, rotated 90° to fit the page. Source: Wikipedia, “Tennis court”; diagram by NielsF et al., Wikimedia Commons, CC BY-SA 3.0. Adaptation: rotation only.

Checkpoint 1C must produce `task1c_tennis_correspondences.png`, `task1c_tennis_insertion.png`, and `task1c_tennis_alignment.png`. The alignment view draws segments between the supplied A–H image anchors in green and your reprojected anchors in orange. The student archives intentionally omit metric answers and reference homographies.

1D. Short analysis [0.6 point]

Report the basketball corner-transfer error and tennis eight-anchor reprojection RMSE from `outputs/task1/task1d_metrics.json`. Can the tennis homography and true court dimensions alone determine a unique camera height when the intrinsic matrix is unknown? Explain the ambiguity in $H \sim K[r_1 \ r_2 \ t]$ and contrast it with the non-coplanar basketball landmarks. “Camera height” means optical center height above the court, not the physical height of the camera operator.

5 Task 2: Camera projection and 3D insertion [2.4 points]

Next we will insert not only a 2D logo but a real 3D object into the scene. A full camera matrix lets us project every vertex of a Stanford bunny from metric world coordinates to the basketball image.

2A. Estimate the camera matrix [1.2 points]

In `task2_projection.py`, implement `estimate_projection_matrix`. Given $N \geq 6$ 3D-to-2D pairs, build the $2N \times 12$ homogeneous DLT system and return a 3×4 matrix P such that

$$\lambda[u, v, 1]^\top = P[X, Y, Z, 1]^\top.$$

Use all eight supplied landmarks. If K is known or constrained, camera pose can be recovered from the metric court plane. Otherwise, non-coplanar points are needed, so we include the four elevated backboard corners.

2B. Reprojection and bunny insertion [0.6 point]

The provided code moves the bunny to center court, raises its lowest vertex to $z = 0$, projects every vertex, and renders it with height-dependent color. Checkpoint 2A produces `task2a_reprojection.png`; Checkpoint 2B produces `task2b_bunny_insertion.png`. Report the mean and maximum landmark reprojection error.

2C. Camera pose [0.6 point]

The provided decomposition computes K , R , t , and C . Include `task2c_camera_pose.png`, report the camera center and height, verify $C = -R^\top t$, and explain why the elevated landmarks resolve the planar degeneracy. Treat these quantities as estimates from an unconstrained, noisy DLT fit rather than surveyed camera calibration. Checkpoint 2C writes them to `task2c_metrics.json`.

6 Task 3: Two-view epipolar geometry [2.4 points]

We now move from one image to two views. Our goal is to describe the geometric relationship between them: a point in one view restricts its match in the other view to an epipolar line. The data provide corresponding points $(u_1, v_1) \leftrightarrow (u_2, v_2)$ such as the color-matched examples below.

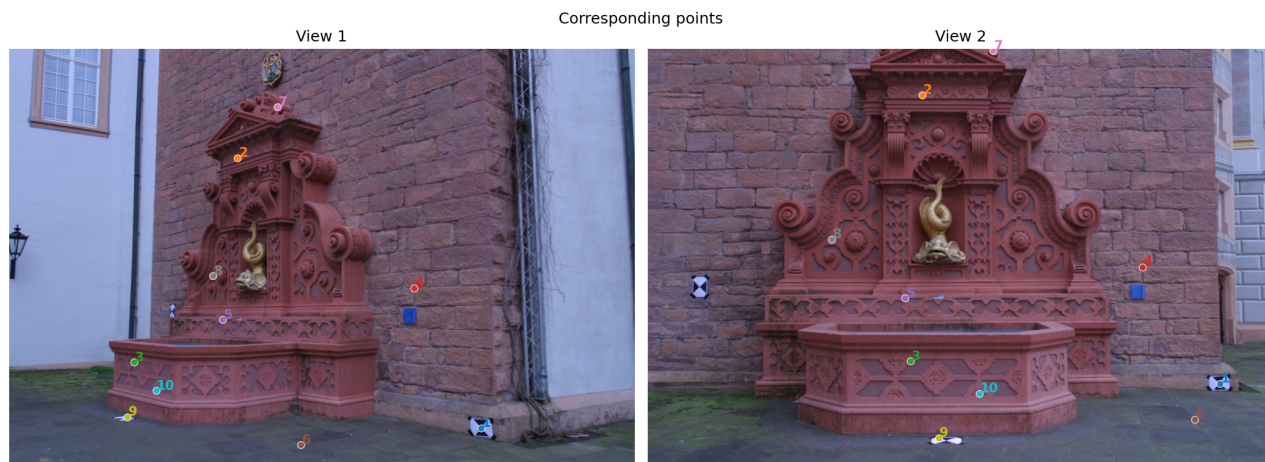


Figure 5: Given point correspondences. Matching colors and numbers identify the same scene point in the two views.

3A. Estimate the fundamental matrix [1.6 points]

In `task3_epipolar.py`, implement `estimate_fundamental_matrix`. Each input row is (u_1, v_1, u_2, v_2) . Normalize the points in each image, form the $N \times 9$ eight-point design matrix, solve with SVD, enforce rank two by setting the smallest singular value to zero, and denormalize. Checkpoint 3A writes your matrix to `task3a_fundamental_matrix.txt` so you can inspect it before continuing.

For ideal, exact correspondences, the model predicts

$$\mathbf{x}_2^\top F \mathbf{x}_1 = 0.$$

Real keypoint locations have detection and rounding error, so do not expect strict floating-point equality for every provided match. Evaluate a geometric point-to-epipolar-line distance instead.

3B. Plot corresponding points and epipolar lines [0.4 point]

Checkpoint 3B first saves `task3b_correspondences.png`. It then uses your F to compute a line in the opposite image for each selected point and saves `task3b_epipolar_lines.png`. Verify visually that each line passes near its corresponding point.

3C. Quantitative check [0.4 point]

Verify that F has rank two. Report the mean and median symmetric epipolar distance over all 200 evaluation matches, and explain why these real-data values are nonzero. Checkpoint 3C writes the values to `task3c_metrics.json`.

7 Optional bonus: foreground-preserving video insertion [+1.0 point]

Gen-AI leverage allowed

A naive broadcast overlay draws the logo after the players and incorrectly covers them. Design a compositor in which dynamic players remain in front of the virtual court logo. No starter or helper function is provided for this bonus. Create and submit `bonus_tennis_video.py`; your code must implement its own video loading, clean-background estimation, soft foreground mask, logo insertion, composition, and output writing. You may reuse your completed Task 1 geometry and alpha-blending functions.

Each clip archive supplies eight A–H image anchors but no metric coordinates or homography. Derive the metric A–H coordinates once in Task 1, then reuse them with each clip’s image anchors.

Run the method on all five supplied videos. Submit exactly one representative composited PNG for each clip: `match001_representative.png`, `match005_representative.png`, `match028_representative.png`, `match091_representative.png`, and `match125_representative.png`. Each frame should visibly demonstrate that a player remains in front of the inserted logo. Also submit one complete `match005_occlusion_aware.mp4`; an MP4 is not required for the other clips.

Questions to guide your design. Can multiple frames reveal the clean court behind moving players? How could you separate changing foreground pixels from that clean background? Once the logo is inserted into the background, how should the original foreground be placed back on top? The implementation choice is open.

8 Report and submission [1.8 points]

Submit your revised copies of all three task files, the generated `outputs/` folders, and one PDF report using `report_template.tex`. Your code submission must include your completed editable blocks in `task1_homography.py`, `task2_projection.py`, and `task3_epipolar.py`; figures or a report alone are not sufficient.

Task	Revised code to submit	Required generated evidence
1	<code>task1_homography.py</code>	two basketball PNGs; three tennis PNGs; metrics JSON
2	<code>task2_projection.py</code>	bunny insertion, reprojection, camera pose; metrics JSON
3	<code>task3_epipolar.py</code>	matrix text; correspondences, epipolar lines; metrics JSON
Bonus	<code>bonus_tennis_video.py</code>	if attempted: five representative PNGs; one match005 MP4

The report should be concise (recommended 3–5 pages), use the three task sections above, and finish with a short limitations/improvements section. Every figure needs a readable caption and must be generated by your run.

9 Integrity and generative-AI policy

This is an individual assignment. You may discuss concepts, but the required functions, metric tennis coordinates, analysis, and report must be your own. Generative AI tools such as GitHub Copilot, Codex, ChatGPT, Claude, and similar systems may not be used for any required task. Generative AI may be used only in the optional bonus section explicitly marked **Gen-AI leverage allowed**. Disclose any bonus use in the report template, including the tool, purpose, and affected files.

Course staff may ask you to explain your implementation or make a small related modification to verify authorship and understanding. Before submission, run each of the three task files from the `handout/` directory and verify that every required image opens and agrees with the values reported in your PDF.